

Topic -5 (Part-1): Monte Carlo Methods (On-Policy)

AIMLCZG512-Deep Reinforcement Learning

Lecture Notes

1. Topics Covered

1. Monte Carlo methods as model-free learning methods based on complete episodes.
2. Episodes, returns, first-visit Monte Carlo prediction, and every-visit Monte Carlo prediction.
3. Numerical estimation of $v_{\pi}(s)$ from sampled returns, including repeated visits to the same state.
4. Why action-value estimates $q_{\pi}(s, a)$ are needed for Monte Carlo control without a model.
5. Exploration in Monte Carlo methods: exploring starts and ϵ -soft stochastic policies.
6. On-policy Monte Carlo control and its relation to generalized policy iteration.
7. Distinction between on-policy and off-policy learning at an introductory level.
8. Review questions involving conceptual, numerical, and modelling exercises.

2. Monte Carlo Methods for Reinforcement Learning

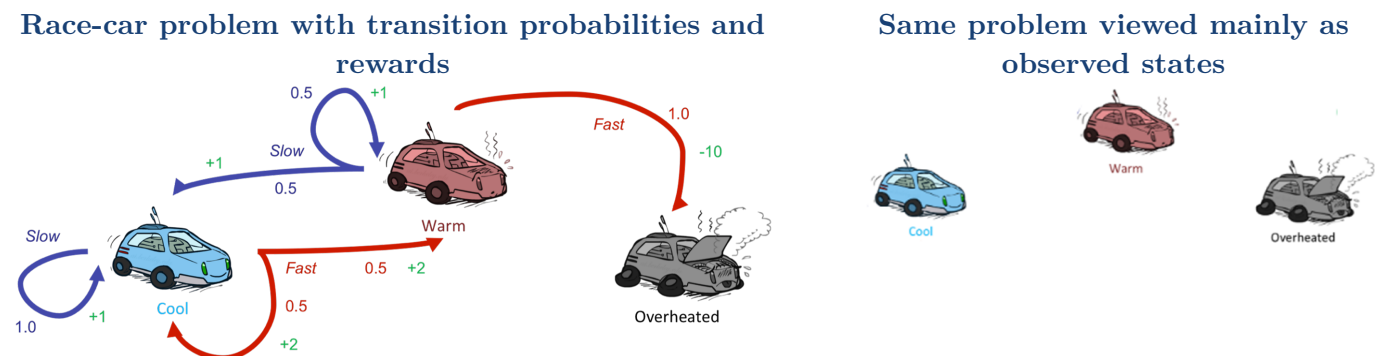
Monte Carlo methods are computational methods that estimate unknown quantities by repeated sampling. In reinforcement learning, the unknown quantities are usually value functions and improved policies. Instead of using the complete transition model $p(s', r \mid s, a)$, Monte Carlo methods use **sampled episodes** generated by interaction with the environment or by simulation.

This is the first major step from planning to learning. In dynamic programming, the agent needed a model and performed expected Bellman backups. In Monte Carlo learning, the agent observes complete sample experience and estimates values by averaging the returns actually obtained after visits to states or state-action pairs.

Central idea. Monte Carlo methods learn from **complete returns**. A state is evaluated by looking at the total discounted reward that followed visits to that state across many episodes. Thus, MC methods are **model-free**: they do not require transition probabilities or expected rewards to be known.

2.1. From Model-Based DP to Data-Driven MC Learning

The race-car problem can be understood in two ways. If the transition probabilities and rewards are known, the problem can be solved by dynamic programming. If the transition model is not available, but the agent can drive many episodes and observe what happens, then Monte Carlo methods can estimate values directly from experience.



With the left diagram, a DP method can compute expected backups because the transition probabilities and rewards are specified. With the right diagram, a Monte Carlo learner may not know the

probabilities. It can still learn by observing episodes such as `Cool -> Warm -> Overheated` or `Cool -> Cool -> Cool` and averaging the returns that follow visits to each state.

Why MC is useful in RL. It makes the problem more data-driven. The learner can estimate values from actual or simulated experience even when the full model is unknown, difficult to write down, or too complicated for exact DP backups.

3. Monte Carlo Prediction

Monte Carlo methods are most naturally introduced for **episodic tasks**. An episode begins from some starting state and ends in a terminal state. Only after the episode is complete do we know the full return following each visited state.

For an episode

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T, S_T,$$

the return following time t is

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T. \quad (1)$$

Here $\gamma \in [0, 1]$ is the discount factor. If γ is close to 1, later rewards still matter significantly. If γ is small, near-term rewards dominate.

3.1. First-Visit and Every-Visit Monte Carlo Prediction

Monte Carlo prediction estimates the state-value function $v_\pi(s)$ for a given policy π :

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s]. \quad (2)$$

The estimate is obtained by averaging observed returns after visits to s .

Method	Meaning
First-visit MC	In each episode, use only the return following the first time state s is visited. Later visits to the same state in the same episode are ignored for that episode.
Every-visit MC	Use the return following every occurrence of state s in the episode. If s occurs three times, three returns are appended.

Both methods converge to $v_\pi(s)$ under suitable conditions as the number of visits grows. First-visit MC is slightly easier to analyse. Every-visit MC uses more samples from each episode.

An illustration. Suppose one episode contains two visits to C and the returns following those visits are 6 and 2. First-visit MC appends only 6 for C from this episode. Every-visit MC appends both 6 and 2. Over many episodes, first-visit MC gives one return per episode for a state, whereas every-visit MC may give multiple, correlated returns from the same episode.

Bias and variance. Under a fixed policy and independent episodes, first-visit returns for a state are clean samples of the expected return after the first visit to that state. This makes first-visit MC easier to analyse and explains why it is often used in textbook algorithms. Every-visit MC also converges, and it uses more returns from each episode; however, repeated returns from the same episode are correlated. In simple terms, first-visit MC is cleaner theoretically, while every-visit MC is often natural when one wants to use all observed visits.

3.2. First-Visit Monte Carlo Prediction Algorithm

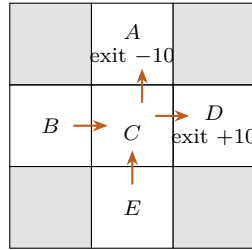
First-Visit Monte Carlo Prediction, for estimating $V \approx v_\pi$

1. **Input:** a policy π to be evaluated.
2. **Initialize:** $V(s)$ arbitrarily for all states s ; create an empty list $Returns(s)$ for every state.
3. **Repeat for each episode:**
 - (a) Generate one complete episode following π .
 - (b) Set $G \leftarrow 0$.
 - (c) Scan the episode backward from $t = T - 1$ to 0:
 - i. Update $G \leftarrow \gamma G + R_{t+1}$.
 - ii. If S_t does not appear earlier in this episode, then:
 - A. Append G to $Returns(S_t)$.
 - B. Set $V(S_t) \leftarrow$ average of all values in $Returns(S_t)$.

Why scanning backward helps. At the end of the episode, the final reward is known. Moving backward allows us to compute each return recursively using $G \leftarrow \gamma G + R_{t+1}$. The value estimate is updated only when the scanned occurrence is the first visit of that state in the original episode.

3.3. Grid Example

Consider the five-state grid below. The policy directs the agent from B to C to D , from E to C , and from C either to D or to A depending on the sampled episode. State D exits with reward $+10$, state A exits with reward -10 , and ordinary moves have reward -1 .



The following four episodes are observed. Here $\gamma = 1$.

Episode	Observed trajectory	First-visit returns appended
1	$B \xrightarrow{-1} C \xrightarrow{-1} D \xrightarrow{+10} T$	$G(B) = 8, G(C) = 9, G(D) = 10$
2	$B \xrightarrow{-1} C \xrightarrow{-1} D \xrightarrow{+10} T$	$G(B) = 8, G(C) = 9, G(D) = 10$
3	$E \xrightarrow{-1} C \xrightarrow{-1} A \xrightarrow{-10} T$	$G(E) = -12, G(C) = -11, G(A) = -10$
4	$E \xrightarrow{-1} C \xrightarrow{-1} D \xrightarrow{+10} T$	$G(E) = 8, G(C) = 9, G(D) = 10$

In the actual algorithm, $V(s)$ is updated as each episode is processed. The running estimates are:

After episode	$V(A)$	$V(B)$	$V(C)$	$V(D)$	$V(E)$
1	–	8	9	10	–
2	–	8	9	10	–
3	-10	8	$\frac{9+9-11}{3} = 2.33$	10	-12
4	-10	8	$\frac{9+9-11+9}{4} = 4$	10	$\frac{-12+8}{2} = -2$

Thus the final estimates are

$$V(A) = -10, \quad V(B) = 8, \quad V(C) = 4, \quad V(D) = 10, \quad V(E) = -2.$$

	A	
	-10	
B	C	D
8	4	10
	E	
	-2	

3.4. Five-Episode Example with Repeated Visits

Now consider a longer episodic task with states A, B, C, D, E, G and terminal state T . State G gives a final reward of $+10$. Other rewards are shown on each transition. We use $\gamma = 0.9$. Some states appear more than once in the same episode, so first-visit and every-visit MC can differ.

The five observed episodes are written below as transition rows. Each arrow shows the reward received on that transition. The final line in each box lists the first-visit returns that are appended to the return list of the corresponding state.

Episode 1

$$A \xrightarrow{r=-1} B \xrightarrow{r=0} C \xrightarrow{r=-1} B \xrightarrow{r=0} D \xrightarrow{r=2} C \\ \xrightarrow{r=0} E \xrightarrow{r=-1} D \xrightarrow{r=0} E \xrightarrow{r=0} G \xrightarrow{r=10} T.$$

First-visit returns: $G(A) = 2.84, G(B) = 4.27, G(C) = 4.75, G(D) = 7.09, G(E) = 6.29, G(G) = 10.$

Episode 2

$$A \xrightarrow{r=0} C \xrightarrow{r=-1} B \xrightarrow{r=0} C \xrightarrow{r=0} D \xrightarrow{r=2} E \\ \xrightarrow{r=0} D \xrightarrow{r=0} B \xrightarrow{r=-1} E \xrightarrow{r=0} G \xrightarrow{r=10} T.$$

First-visit returns: $G(A) = 3.81, G(C) = 4.23, G(B) = 5.81, G(D) = 7.18, G(E) = 5.75, G(G) = 10.$

Episode 3

$$B \xrightarrow{r=-1} A \xrightarrow{r=0} B \xrightarrow{r=0} C \xrightarrow{r=-1} D \xrightarrow{r=2} C \\ \xrightarrow{r=0} E \xrightarrow{r=0} D \xrightarrow{r=0} E \xrightarrow{r=0} G \xrightarrow{r=10} T.$$

First-visit returns: $G(B) = 3.46, G(A) = 4.95, G(C) = 6.11, G(D) = 7.90, G(E) = 7.29, G(G) = 10.$

Episode 4

$$C \xrightarrow{r=-1} B \xrightarrow{r=0} C \xrightarrow{r=-1} B \xrightarrow{r=0} A \xrightarrow{r=0} D \\ \xrightarrow{r=2} E \xrightarrow{r=0} D \xrightarrow{r=0} E \xrightarrow{r=0} G \xrightarrow{r=10} T.$$

First-visit returns: $G(C) = 3.25, G(B) = 4.72, G(A) = 7.70, G(D) = 8.56, G(E) = 7.29, G(G) = 10.$

Episode 5

$$A \xrightarrow{r=-1} B \xrightarrow{r=0} D \xrightarrow{r=2} B \xrightarrow{r=0} C \xrightarrow{r=-1} E \\ \xrightarrow{r=0} D \xrightarrow{r=0} E \xrightarrow{r=0} C \xrightarrow{r=0} G \xrightarrow{r=10} T.$$

First-visit returns: $G(A) = 3.84, G(B) = 5.38, G(D) = 5.97, G(C) = 4.90, G(E) = 6.56, G(G) = 10.$

The first-visit estimates are updated after each episode:

After episode	$V(A)$	$V(B)$	$V(C)$	$V(D)$	$V(E)$	$V(G)$
1	2.84	4.27	4.75	7.09	6.29	10.00
2	3.33	5.04	4.49	7.14	6.02	10.00
3	3.87	4.51	5.03	7.39	6.44	10.00
4	4.83	4.56	4.58	7.68	6.66	10.00
5	4.63	4.73	4.65	7.34	6.64	10.00

The final comparison is:

State	First-visit MC estimate	Every-visit MC estimate
A	$(2.84 + 3.81 + 4.95 + 7.70 + 3.84)/5 = 4.63$	4.63
B	$(4.27 + 5.81 + 3.46 + 4.72 + 5.38)/5 = 4.73$	average of 10 returns = 5.40
C	$(4.75 + 4.23 + 6.11 + 3.25 + 4.90)/5 = 4.65$	average of 10 returns = 5.62
D	$(7.09 + 7.18 + 7.90 + 8.56 + 5.97)/5 = 7.34$	average of 10 returns = 7.47
E	$(6.29 + 5.75 + 7.29 + 7.29 + 6.56)/5 = 6.64$	average of 10 returns = 7.73
G	10.00	10.00

First-visit MC treats each episode as contributing at most one return for a state. **Every-visit MC** gives repeated states additional influence because each occurrence contributes a return.

3.5. Discussion: Applicability of MC Prediction

Monte Carlo prediction is especially useful when the task naturally produces complete episodes and the state information available to the learner is clear enough to group repeated visits to the same state. If the same observed state actually mixes many hidden situations, then the estimated value may average together different cases. When the state is well defined, MC prediction can estimate values directly from experience without knowing transition probabilities.

MC prediction is attractive in simulation, games, robot trials, repeated service processes, and operational settings where full episodes can be generated many times. Its main limitation is that it waits until the end of an episode before updating. Therefore it is less natural for continuing tasks without episode endings and may learn slowly when important rewards are delayed far into the future.

4. From State Values to Action Values

For prediction, estimating $v_\pi(s)$ is sufficient if the policy is fixed. For control, however, the agent must choose actions. With a known model, state values can be used for action selection through one-step lookahead:

$$\pi(s) \in \arg \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma V(s')]. \quad (3)$$

This is how dynamic programming can improve a policy using $V(s)$ and the model.

In Monte Carlo control, the model is not assumed known. Therefore, the agent cannot compute the above expectation for every action. It is more useful to estimate the **action-value function**

$$q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]. \quad (4)$$

Then actions can be selected directly by comparing $Q(s, a)$ values.

Visit to a state-action pair. The pair (s, a) is visited at time t if $S_t = s$ and $A_t = a$. In Monte Carlo action-value estimation, returns are averaged after visits to state-action pairs, not merely after visits to states.

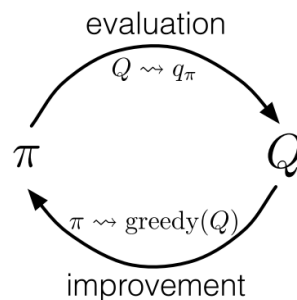
A new difficulty now appears. Some state-action pairs may not be visited at all under the current policy. If the agent never tries action a in state s , it cannot estimate $q_\pi(s, a)$ from experience. This is the **maintaining exploration** problem.

Exploration approach	Meaning
Exploring starts	Every state-action pair has nonzero probability of being selected as the starting state and starting action of an episode. This assumption guarantees coverage in theory, but it is often unrealistic because we cannot freely start real systems from any desired state-action pair.
Stochastic policies	The policy keeps a nonzero probability of selecting all actions in each visited state. This is more practical for on-policy MC control because exploration is maintained during normal behaviour.

Key point. Maintaining exploration is not optional in MC control. If an action is never tried in a state, its action value cannot be estimated from data. Exploring starts solve this mathematically; ϵ -soft policies solve it more practically by keeping all actions possible.

5. Monte Carlo Control and Generalized Policy Iteration

Monte Carlo control follows the same broad idea as **generalized policy iteration** (GPI). There are two interacting processes. One process estimates the action-value function of the current policy. The other process improves the policy by making it more greedy with respect to the current action-value estimates.



Monte Carlo control as generalized policy iteration using action values.

The important comparison with dynamic programming is this. In DP, if a model is known, policy improvement may use $V(s)$ together with one-step lookahead, or it may use $Q(s, a)$ directly if action values are available. In Monte Carlo control, the model is not assumed known; therefore $Q(s, a)$ is normally estimated from sampled returns so that actions can be compared without transition probabilities.

- In **DP evaluation**, values are computed from the model using expected backups over all possible next states and rewards.
- In **MC evaluation**, values are estimated by averaging complete returns observed in sampled episodes.
- In **DP improvement**, the policy can be made greedy using the model-based one-step lookahead.

- In **MC improvement**, the policy is improved with respect to $Q(s, a)$ because the action values already summarize the observed return after taking action a in state s .
- In on-policy MC control, the policy is moved only toward an ϵ -greedy policy so that exploration is maintained.

Thus, MC control is a model-free form of the evaluation-improvement cycle. It retains the GPI logic from DP, but replaces exact expected backups by episode-based averaging of returns.

6. On-Policy First-Visit Monte Carlo Control

On-policy control means that the same policy is used to generate episodes and is also the policy being evaluated and improved. An episode is generated using the current ϵ -soft policy. After the episode is processed, $Q(s, a)$ and the policy probabilities are updated for visited state-action pairs. The next episode is then generated using this revised policy. Thus the behaviour policy and the target policy are not separated.

For a finite action set $\mathcal{A}(s)$, an ϵ -greedy improvement step produces an ϵ -soft policy:

$$\pi(a | s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{if } a = A^*(s), \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{if } a \neq A^*(s), \end{cases} \quad (5)$$

where

$$A^*(s) \in \arg \max_a Q(s, a). \quad (6)$$

On-policy first-visit MC control (for ϵ -soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\epsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ϵ -soft policy

$Q(s, a) \in \mathbb{R}$ arbitrarily, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T - 1, T - 2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg \max_a Q(S_t, a)$, with ties broken arbitrarily

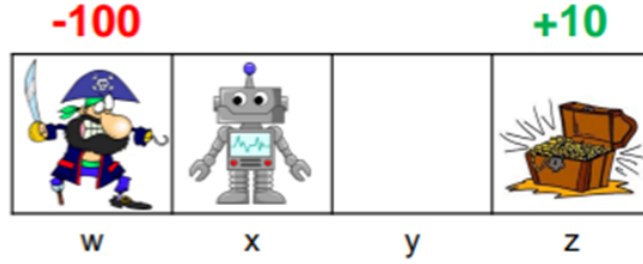
For all $a \in \mathcal{A}(S_t)$:

$$\pi(a | S_t) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(S_t)|, & \text{if } a = A^*, \\ \epsilon/|\mathcal{A}(S_t)|, & \text{if } a \neq A^*. \end{cases}$$

6.1. Example 1: Four-State Line World

The line-world example has states w, x, y, z , discount factor $\gamma = 0.9$, a bad terminal reward -100 at the left, and a good terminal reward $+10$ at the right. The two main nonterminal decision states are x and y . State w has the available action *Exit* leading to the bad terminal, and state z has the available action *Exit* leading to the good terminal.

$$\gamma = 0.9$$



Line-world control example: moving toward z is desirable; moving to w is dangerous.

Let $\epsilon = 0.2$ for the ϵ -greedy improvement step. Initially the policy is some ϵ -soft policy. The following episodes illustrate the algorithm sequentially: each episode is generated using the latest policy, and after the episode the policy is updated for the visited state-action pairs.

Episode 1

Observed: $x \xrightarrow{R,0} y \xrightarrow{R,0} z \xrightarrow{Exit,+10} T$.

Backward processing gives $G(z, Exit) = 10$, $G(y, R) = 0 + 0.9(10) = 9$, and $G(x, R) = 0 + 0.9(9) = 8.1$. Hence $Q(z, Exit) = 10$, $Q(y, R) = 9$, and $Q(x, R) = 8.1$. The improved policy becomes $\pi(R | y) = 0.9$, $\pi(L | y) = 0.1$, and $\pi(R | x) = 0.9$, $\pi(L | x) = 0.1$.

Episode 2

Observed: $x \xrightarrow{L,0} w \xrightarrow{Exit,-100} T$.

The return for $(w, Exit)$ is -100 , and the return for (x, L) is $0 + 0.9(-100) = -90$. Thus $Q(w, Exit) = -100$ and $Q(x, L) = -90$. Since $Q(x, R) = 8.1$ is still larger, the policy at x remains $\pi(R | x) = 0.9$, $\pi(L | x) = 0.1$.

Episode 3

Observed: $y \xrightarrow{L,0} x \xrightarrow{R,0} y \xrightarrow{R,0} z \xrightarrow{Exit,+10} T$.

First-visit returns are $G(y, L) = 0 + 0.9(0) + 0.9^2(0) + 0.9^3(10) = 7.29$, $G(x, R) = 0 + 0.9(0) + 0.9^2(10) = 8.1$, $G(y, R) = 9$, and $G(z, Exit) = 10$. The averages keep $Q(y, R) = 9$ and give $Q(y, L) = 7.29$, so the policy at y remains biased toward R .

Episode 4

Observed: $x \xrightarrow{R,0} y \xrightarrow{L,0} x \xrightarrow{R,0} y \xrightarrow{R,0} z \xrightarrow{Exit,+10} T$.

For first-visit control, the first occurrence of (x, R) has return $0 + 0.9(0) + 0.9^2(0) + 0.9^3(0) + 0.9^4(10) = 6.56$. The first occurrence of (y, L) has return 7.29 , (y, R) has return 9 , and $(z, Exit)$ has return 10 . After averaging, $Q(x, R) = (8.1 + 8.1 + 6.56)/3 = 7.59$, $Q(y, R) = 9$, and $Q(y, L) = 7.29$.

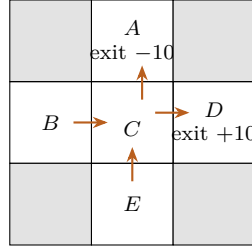
The resulting greedy choices are R at x and R at y ; the only available actions at w and z are $Exit$ and $Exit$, respectively. Therefore, with $\epsilon = 0.2$ and two available actions in each nonterminal decision state,

$$\pi(R | x) = 0.9, \quad \pi(L | x) = 0.1, \quad \pi(R | y) = 0.9, \quad \pi(L | y) = 0.1.$$

Exam interpretation. Sometimes a question may directly give several episodes together. In that case, unless otherwise stated, treat each new episode as having been generated after the policy update from the previous episode. The purpose is to test return computation, Q averaging, and ϵ -greedy policy improvement.

6.2. Example 2: Grid Control Example

Consider again the five-state grid. State D exits with reward $+10$, state A exits with reward -10 , and ordinary moves have reward -1 . For this simplified control demonstration, the displayed actions are $B \xrightarrow{E} C$, $E \xrightarrow{N} C$, and from C either E toward D or N toward A .



Run on-policy first-visit MC control with $\gamma = 0.9$ and $\epsilon = 0.2$. The following episode sequence illustrates the continual update of Q and π .

Episode 1

Observed: $B \xrightarrow{E, -1} C \xrightarrow{E, -1} D \xrightarrow{Exit, +10} T$.

Returns: $G(C, E) = -1 + 0.9(10) = 8$, and $G(B, E) = -1 + 0.9(8) = 6.2$. Now $Q(C, E) = 8$, so the improved policy at C gives $\pi(E | C) = 0.9$ and $\pi(N | C) = 0.1$.

Episode 2

Observed: $E \xrightarrow{N, -1} C \xrightarrow{N, -1} A \xrightarrow{Exit, -10} T$.

Returns: $G(C, N) = -1 + 0.9(-10) = -10$, and $G(E, N) = -1 + 0.9(-10) = -10$. Since $Q(C, E) = 8$ and $Q(C, N) = -10$, the policy at C remains $\pi(E | C) = 0.9$, $\pi(N | C) = 0.1$.

Episode 3

Observed: $E \xrightarrow{N, -1} C \xrightarrow{E, -1} D \xrightarrow{Exit, +10} T$.

Returns: $G(C, E) = 8$, and $G(E, N) = 6.2$. The estimate $Q(C, E)$ remains 8, while $Q(E, N) = (-10 + 6.2)/2 = -1.9$.

Episode 4

Observed: $B \xrightarrow{E, -1} C \xrightarrow{E, -1} D \xrightarrow{Exit, +10} T$.

Returns: $G(C, E) = 8$, and $G(B, E) = 6.2$. Now $Q(B, E) = 6.2$, $Q(E, N) = -1.9$, $Q(C, E) = 8$, and $Q(C, N) = -10$.

The updated action-value estimates and policy are:

Pair	Returns averaged	Q	Policy implication
(B, E)	6.20, 6.20	6.20	Only displayed action from B ; choose E .
(E, N)	-10.00, 6.20	-1.90	Only displayed action from E ; choose N .
(C, E)	8, 8, 8	8.00	Greedy action at C .
(C, N)	-10	-10.00	Nongreedy action at C .

Thus, for the two actions at C ,

$$\pi(E \mid C) = 1 - \epsilon + \frac{\epsilon}{2} = 0.9, \quad \pi(N \mid C) = \frac{\epsilon}{2} = 0.1.$$

The next episode will therefore usually move from C toward D , but occasional movement toward A is still possible for exploration.

Important point. MC control improves the policy only through state-action pairs that are sampled. The policy update is made whenever a first-visit return updates a relevant $Q(s, a)$ value. The updated policy affects future episodes, not the already completed episode.

7. On-Policy and Off-Policy Learning

In Monte Carlo control, the policy used to collect episodes matters. Sutton and Barto separate two cases: **on-policy** methods and **off-policy** methods.

On-policy learning evaluates or improves the same policy that is used to make decisions. In the algorithm above, the current policy is ϵ -soft. It generates the episode, its returns are used to update $Q(s, a)$, and the same policy is then shifted toward the ϵ -greedy policy with respect to the updated Q . Thus, the learner improves the behaviour it is actually following.

Off-policy learning separates the two roles. A **behaviour policy** generates exploratory data, while a different **target policy** is evaluated or improved. This is useful when we want to learn about a greedy or near-optimal target policy while still behaving exploratorily enough to collect data.

Type	Policy being learned	Data-generating policy
On-policy	The same policy that is used for action selection is evaluated and improved.	Episodes are generated by the current policy, usually with exploration built in, such as an ϵ -soft policy.
Off-policy	A target policy is evaluated or improved; it may be greedy or closer to optimal.	A separate behaviour policy generates episodes and must explore enough to cover the target policy's actions.

Why off-policy is more delicate. A return observed while following one policy is not automatically an unbiased return for another policy. Therefore off-policy Monte Carlo methods usually need correction terms, such as importance sampling, to account for the mismatch between the behaviour policy and the target policy. This gives flexibility, but can increase variance.

8. Review of Key Terms

Term	Short definition
Monte Carlo method	A method that estimates quantities by averaging outcomes from sampled experience.
Model-free learning	Learning without requiring explicit transition probabilities or reward dynamics.
Episode	A complete trajectory from a start state to a terminal state.
Return	The discounted sum of rewards following a time step.
First-visit MC	Uses only the first occurrence of a state or state-action pair in each episode.
Every-visit MC	Uses every occurrence of a state or state-action pair in each episode.
Action-value function	$q_\pi(s, a)$, the expected return after taking action a in state s and then following π .
Exploring starts	An assumption that every state-action pair can be selected as the starting state-action pair with nonzero probability.
ϵ -soft policy	A stochastic policy satisfying $\pi(a s) \geq \epsilon/ \mathcal{A}(s) $ for every action in each state.
On-policy learning	Evaluating or improving the same policy that is used to generate behaviour.
Off-policy learning	Evaluating or improving a target policy while data are generated by a different behaviour policy.

9. Review Questions

1. Explain how Monte Carlo methods differ from dynamic programming methods in terms of model requirement, backup structure, and use of complete episodes.
2. Define the return G_t for an episodic task. For rewards $R_{t+1} = 2$, $R_{t+2} = -1$, $R_{t+3} = 5$ and $\gamma = 0.9$, compute G_t .
3. Distinguish first-visit MC and every-visit MC. Construct a short episode in which the two methods append different numbers of returns for the same state.
4. In the five-state grid example, recompute the running estimate of $V(C)$ after each of the four displayed episodes when $\gamma = 1$.
5. In the repeated-visit example, explain why $V(B)$ differs under first-visit MC and every-visit MC. What does every-visit MC count that first-visit MC ignores?
6. A robot can be in states Safe, Risky, and Goal. It follows a fixed policy for 100 episodes. Describe precisely how first-visit MC prediction would estimate $V(\text{Risky})$.
7. Why are state values alone sufficient for action selection in DP when a model is known, but not sufficient in model-free Monte Carlo control?
8. Define a visit to a state-action pair. Why is maintaining exploration more serious for action-value estimation than for state-value estimation?
9. In the line-world example, compute $G(x, \text{Right})$ for $x \xrightarrow{\text{Right}, 0} y \xrightarrow{\text{Right}, 0} z \xrightarrow{\text{Exit}, +10} T$ with $\gamma = 0.9$. Compute $G(x, \text{Left})$ for $x \xrightarrow{\text{Left}, 0} w \xrightarrow{\text{Exit}, -100} T$.
10. Suppose $|\mathcal{A}(s)| = 4$ and $\epsilon = 0.2$. Under an ϵ -greedy improvement step that produces an ϵ -soft policy, what probability is assigned to the greedy action and to each nongreedy action?

11. In a state s , assume $Q(s, a_1) = 4.0$, $Q(s, a_2) = 6.0$, and $Q(s, a_3) = 5.0$. For $\epsilon = 0.3$, write the improved ϵ -soft policy over the three actions.
12. Explain how on-policy first-visit MC control fits the generalized policy iteration idea. Identify the evaluation part and the improvement part.
13. A recommender system shows one of four content categories to a user during a session and observes session-level reward only at the end. Formulate this as an episodic MC control problem by identifying states, actions, rewards, episodes, and the reason an ϵ -soft policy may be useful.
14. A simulator generates episodes under an exploratory policy, but the designer wants to evaluate a greedy target policy. Explain why this is an off-policy setting and why correction for the policy mismatch is needed.
15. In the grid-control example, explain why the updated policy at C favours action E even though action N is still assigned nonzero probability.

10. Points to Remember

- MC methods learn from **complete returns**, not from one-step expected backups.
- They are **model-free**: no transition probability model is required.
- First-visit and every-visit MC differ in how they treat repeated visits within the same episode.
- For control without a model, action values $Q(s, a)$ are usually more useful than state values $V(s)$.
- On-policy MC control uses the same exploratory policy for acting, evaluation, and improvement.
- ϵ -soft policies maintain exploration by assigning every action a nonzero probability.

11. Required Reading

Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. Required reading: Chapter 5, especially Sections 5.1 to 5.4, covering Monte Carlo prediction, action-value estimation, Monte Carlo control, and on-policy control without exploring starts. Section 5.5 may be read for an introductory view of off-policy prediction.

12. References

Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Cambridge, MA: MIT Press.

13. Acknowledgements

These lecture notes are based on and adapted from Chapter 5 of Sutton and Barto's *Reinforcement Learning: An Introduction*, Second Edition.

End of Topic -5 (Part-1)